

Laboratorio 10 (A-L)

Programmazione Funzionale e Lambda, Ereditarietà e Specializzazione,
Progettazione Architettuale e Polimorfismo, Gestione dello Stato

Michele Gentile

Maggio 2026

1 Processore di Testo Funzionale

1. Si definisca un'interfaccia funzionale personalizzata **Trasformatore** annotata con `@FunctionalInterface` che dichiari un unico metodo `String trasforma(String x)`.
2. Si crei una classe **EditorDiTesto** che dispone di un metodo statico `elabora(String s, Trasformatore t)` il quale restituisce il risultato della trasformazione applicata alla stringa di input.
3. Nel metodo `main`, utilizzare le **espressioni lambda** per richiamare il metodo `elabora` ed eseguire le seguenti operazioni:
 - Convertire la stringa interamente in maiuscolo.
 - Aggiungere un prefisso "LOG: " alla stringa originale.
 - Invertire l'ordine dei caratteri della stringa (es. "abc" → "cba").
4. Si integri l'uso dell'interfaccia standard `java.util.function.Predicate<String>` per creare un metodo `elaboraSeValido(String s, Trasformatore t, Predicate<String> p)` che esegua la trasformazione solo se la stringa soddisfa una determinata condizione (es. lunghezza minima).

🔗 Obiettivo: Interfacce Funzionali e Lambda Expression

L'obiettivo è comprendere come passare comportamenti come parametri attraverso le interfacce funzionali, riducendo la verbosità del codice rispetto alle classi anonime.

💡 Tip: Riutilizzo del Codice

Le interfacce funzionali standard fornite dal pacchetto `java.util.function` (come `Function`, `Predicate`, `Consumer`) coprono la maggior parte dei casi d'uso comuni: cercate di usarle prima di definirne di nuove!

⚠ Attenzione: Scope delle variabili

Ricordate che all'interno di una lambda expression potete accedere alle variabili locali del metodo circostante solo se queste sono **final** o *effectively final* (ovvero non vengono mai riassegnate).

2 Cronometro Digitale & Cronometro con Millisecondi

1. Si crei una classe **Cronometro** costruita con un intero che rappresenta i secondi massimi di conteggio. Inizialmente il cronometro è a zero.
2. La classe deve disporre di un metodo `tic()` che incrementa il tempo di un secondo (senza superare il massimo) e di un metodo `toString()` che restituisca il tempo nel formato "[SS/MAX]" (es. se sono passati 5 secondi su 60, la stringa sarà "[05/60]").
3. Si crei una sottoclasse **CronometroPreciso** che aggiunga la gestione dei millisecondi. Il metodo `tic()` in questa classe incrementa il tempo di 100ms.
4. Sovrascrivere il metodo `toString()` affinché mostri anche i millisecondi in coda, ad esempio: "[05/60] : 300ms".

🔗 Obiettivo: Estensione di classi e Overriding

L'obiettivo è comprendere come estendere il comportamento di una classe base aggiungendo attributi e specializzando i metodi esistenti tramite l'overriding.

💡 Tip: Formattazione Stringhe

Per ottenere numeri con lo zero iniziale (es. "05" invece di "5"), potete utilizzare il metodo `String.format("%02d", valore)`.

3 Sistema di Gestione Attuatori

1. Progettare la classe **DispositivoDomo** che rappresenti un generico dispositivo smart con un nome e uno stato (acceso/spento).
2. Modellare una generica **Azione** che disponga di un metodo `esegui(DispositivoDomo d)`.
3. Implementare i seguenti tipi di azione:
 - **Accensione**: imposta lo stato del dispositivo su "acceso".
 - **Spegnimento**: imposta lo stato del dispositivo su "spento".
 - **InvertiStato**: cambia lo stato attuale (da acceso a spento e viceversa).
 - **LogStato**: stampa a video il nome del dispositivo e il suo stato attuale.
4. Aggiungere alla classe **DispositivoDomo** un metodo che permetta di visualizzare lo storico delle azioni ricevute (nell'ordine in cui sono state eseguite).

🔗 Obiettivo: Polimorfismo e Disaccoppiamento

Modellare il comportamento (Azioni) separatamente dai dati (Dispositivi) permette di estendere il sistema con nuove funzionalità senza modificare il codice esistente.

4 Simulatore di Officina Meccanica

1. Progettare una classe **Componente** con un nome e un costo base. Creare tre tipi di componenti: **Pneumatico**, **FiltroOlio**, **PastigliaFreno**.
2. Progettare la classe **Officina** costruita con un intero M che determina la capacità massima del magazzino componenti.
3. La classe deve prevedere i seguenti metodi:
 - **rifornisci()**: riempie il magazzino con M componenti scelti casualmente tra i tre tipi disponibili.
 - **applicaComponente()**: dato un indice, restituisce il componente corrispondente, lo rimuove dal magazzino e lo aggiunge al "conto finale".
 - **getTotale()**: restituisce la somma dei costi dei componenti applicati finora.
 - **emettiFattura()**: stampa l'elenco dei componenti usati e azzera il totale per il prossimo cliente.

🔗 Obiettivo: Interazione tra Oggetti e Gestione Stato

Esercitarsi sulla gestione di array di oggetti, cicli e logica di aggiornamento dei parametri interni di una classe coordinatrice.

⚠️ Attenzione: Gestione Magazzino

Fate attenzione a gestire correttamente gli indici dell'array quando un componente viene "rimosso" o prelevato, per evitare `NullPointerException`.

5 Il Percorso ad Ostacoli

1. Progettare un simulatore di percorso modellando:
 - Il **Robot**, che mantiene la sua posizione (un intero) e un livello di **Energia**. Implementa il metodo `muovi(int passi)`.
 - Il **Percorso** come una sequenza di N caselle. La classe deve gestire il movimento del Robot: se il Robot supera l'ultima casella, ricomincia dall'inizio (percorso circolare).
2. Implementare diversi tipi di caselle:
 - **CasellaPiana**: non ha alcun effetto sul Robot.
 - **CasellaRicarica**: aumenta l'energia del Robot di un valore K .
 - **CasellaFango**: dimezza la distanza percorsa nel prossimo turno dal Robot.

- **CasellaTeletrasporto:** sposta il Robot in una posizione casuale del percorso.
3. Implementare la classe **Simulazione** che inizializza il percorso e fa eseguire 10 turni al Robot, stampando la posizione e l'energia ad ogni passaggio.

⌕ Obiettivo: Gerarchie di Specializzazione

Utilizzare l'ereditarietà per definire diversi comportamenti di risposta (le caselle) a un evento comune (il passaggio del Robot).